

DNNFusion: Accelerating Deep Neural Networks Execution with Advanced Operator Fusion

Wei Niu
William & Mary, USA
wniu@email.wm.edu

Jiexiong Guan
William & Mary, USA
jguan@email.wm.edu

Yanzhi Wang
Northeastern University, USA
yanz.wang@northeastern.edu

Gagan Agrawal
Augusta University, USA
gagrawal@augusta.edu

Bin Ren
William & Mary, USA
bren@wm.edu

Abstract

Deep Neural Networks (DNNs) have emerged as the core enabler of many major applications on mobile devices. To achieve high accuracy, DNN models have become increasingly deep with hundreds or even thousands of operator layers, leading to high memory and computational requirements for inference. Operator fusion (or kernel/layer fusion) is key optimization in many state-of-the-art DNN execution frameworks, such as TensorFlow, TVM, and MNN, that aim to improve the efficiency of the DNN inference. However, these frameworks usually adopt fusion approaches based on certain patterns that are too restrictive to cover the diversity of operators and layer connections, especially those seen in many extremely deep models. Polyhedral-based loop fusion techniques, on the other hand, work on a low-level view of the computation without operator-level information, and can also miss potential fusion opportunities. To address this challenge, this paper proposes a novel and extensive loop fusion framework called DNNFusion. The basic idea of this work is to work at an operator view of DNNs, but expand fusion opportunities by developing a classification of both individual operators and their combinations. In addition, DNNFusion includes 1) a novel mathematical-property-based graph rewriting framework to reduce evaluation costs and facilitate subsequent operator fusion, 2) an integrated fusion plan generation that leverages the high-level analysis and accurate light-weight profiling, and 3) additional optimizations during fusion code generation. DNNFusion is extensively evaluated on 15 DNN models with varied types of tasks, model sizes, and layer counts. The evaluation results demonstrate that DNNFusion finds up to 8.8× higher fusion opportunities, outperforms four state-of-the-art DNN execution frameworks with 9.3× speedup. The memory requirement reduction and speedups can enable the execution of many of the target models on mobile devices and even make them part of a real-time application.

CCS Concepts: • **Software and its engineering → Source code generation**; • **Computing methodologies → Neural networks**; • **Human-centered computing → Mobile computing**.

Keywords: Compiler Optimization, Operator Fusion, Deep Neural Network, Mobile Devices

ACM Reference Format:
Wei Niu, Jiexiong Guan, Yanzhi Wang, Gagan Agrawal, and Bin Ren. 2021. DNNFusion: Accelerating Deep Neural Networks Execution with Advanced Operator Fusion. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI '21), June 20–25, 2021, Virtual, Canada.* ACM, New York, NY, USA, 16 pages. https://doi.org/10.1145/3453483.3454083

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
PLDI '21, June 20–25, 2021, Virtual, Canada
© 2021 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-8391-2/21/06…$15.00
https://doi.org/10.1145/3453483.3454083

1 Introduction

The past ten years have witnessed a resurgence of Machine Learning, specifically in the form of Deep Learning. Deep Neural Networks (DNNs) such as Convolution Neural Networks (CNN) and Recurrent Neural Networks (RNN) serve as the state-of-the-art foundation and core enabler of many applications that have only emerged within the last few years and yet have become extremely popular among all users of computing today [6, 58]. Behind the success of deep learning are the increasingly large model sizes and complex model structures that require tremendous computation and memory resources [16]. There is a difficult trade-off between increasing complexity of DNNs (required for increasing accuracy) and deployment of these DNNs on resource-constrained mobile devices (required for wider reach).

In recent years, there has been a significant emphasis on optimizing the execution of large DNNs. Operator fusion (or *kernel/layer* fusion) has been a common approach towards

improving efficiency of DNN execution [1, 12]. The basic idea of such fusion is the same as the traditional loop fusion done by optimizing compilers [31, 32, 44], and they lead to the following benefits: (i) eliminating unnecessary materialization of intermediate results, (ii) reducing unnecessary scans of the input; and (iii) enabling other optimization opportunities. Traditional end-to-end frameworks like TensorFlow Lite [1], TVM [12], MNN [29], and Pytorch-Mobile [48] all have operator fusion optimizations, which are broadly based on recognizing certain fusion patterns. These transformations have generally been based on a representation called *computational graph* [12], which views the application as a set of operations on tensors, and representation of dependencies in the form of consumption of tensor(s) output by an operation by another operation.

In this paper, we observe that the fusion patterns considered in the past work [1, 12] are too restricted to cover the diversity of operators and layer connections that are emerging. For example, ONNX (Open Neural Network Exchange) [47] lists 167 distinct operators, and creating fusion patterns based on their combinations is unlikely to be a feasible approach. At the same time, traditional compiler loop transformations (including fusion [32, 44]) work on a low-level view of the computation, i.e., loop (indices) and dependence between array elements. More recent work on loop fusion has been based on polyhedral analysis [10], with several different resulting algorithms [2, 3, 9]. Polyhedral analysis, while providing an excellent foundation to rigorously reason the legality of, and explore the space of, loop transformations, can be an "overkill" to capture the relatively simple data structures (tensors) and operations (without loop-carried dependencies) in DNNs. Moreover, polyhedral analysis is normally limited to affine-loop analysis and transformations (although latest efforts [61, 71, 72] do extend it to certain non-affine loop optimizations), and cannot capture certain operation (combinations) in DNNs. An example will be a combination of `Gather`, which copies input to output indirectly using an index array followed by `Flatten`, which changes the dimensionality of a tensor. Finally, the operator view in computational graphs can enable us to exploit properties of these computations, which may be lost when a lower-level view of the computation is considered.

This paper presents DNNFusion, a rigorous and extensive loop fusion framework that can exploit the operator view of computations in DNNs, and yet can enable a set of advanced transformations. The core idea is to classify operators into different types, and develop rules for different combinations of the types, as opposed to looking for patterns with specific combination of operations. Particularly, we first classify the existing operations in a DNN into several groups based on the mapping between their input and output , such as One-to-One, One-to-Many, and others. We also enhance the computational graph representation into the Extended Computational Graph (ECG) representation, where the type (and other properties) of the operation are explicitly noted. Then, we design a mapping type analysis to infer the profitability of fusing operations of different combinations of these types of operators, binning the combination into three groups: likely profitable (and legal), likely not profitable, and ones where profitability may need to be determined through profile information.

Table 1. An empirical study to motivate this work: The relation of overall computation, layer count, and execution efficiency of multiple DNNs. Results are collected on Qualcomm Adreno 650 GPU with an optimized baseline framework with fixed-pattern operator fusion that outperforms all state-of-the-art DNN execution frameworks (called `OurB+` and will be introduced later).

Model	#Total layer	IR size	#FLOPS	Speed (FLOPs/S)
VGG-16 [62]	51	161M	31.0B	320G
YOLO-V4 [7]	398	329M	34.6B	135G
DistilBERT [60]	457	540M	35.3B	78G
MobileBERT [65]	2,387	744M	17.6B	44G
GPT-2 [55]	2,533	1,389M	69.1B	62G

Next, on the ECG representation, we apply a series of *graph rewriting rules* that we have developed. These rules exploit the mathematical properties of the operations and have a similar flavor to the classical optimization called *strength reduction* [14]. Unlike traditional compiler work, however, we apply these rules on operations on tensors (and not scalars) and our set of rules go well beyond the traditional ones. The rest of our framework comprises algorithms for determining fusion of specific operations (based on certain heuristics) and generating optimized fused code. Almost each fusion generates a new operator (and its implementation) that is not present in the original library; however, once a new operator is generated, its implementation can be reused when the same pattern is detected in the same or a different model. Overall, we show that an operator view of the DNN can enable rigorous optimizations, beyond what will be possible with a lower-level view of the computation or the existing (simplistic) work on applying a small set of fusion patterns on the operator view.

In summary, this paper makes the following contributions:

- It designs high-level abstractions (including mapping type analysis and ECG) for operator fusion by leveraging high-level DNN operator information. The approach can handle a diversity of operators and yet enable aggressive optimizations.
- It proposes a novel mathematical-property-based graph rewriting to simplify ECG structure, optimize DNN computations, and facilitate subsequent fusion plan generation.
- It presents an integrated fusion plan generation by combining the benefit of efficient machine-independent mapping type analysis while leveraging a profiling result database.
- It implements optimized fusion code generation, integrating the approach into a state-of-the-art end-to-end DNN

Table 2. Classification of DNN operators in mapping types. These operators are defined in ONNX [47].

Mapping type	Operators	Representative
One-to-One	Add, Asin, BatchNormalization, Cast, Ceil, Clip, Concat, Cos, Erf, Exp, Greater, LeakyRelu, Log, Not, PRelu, Reciprocal, Relu, Round, Sigmoid, Sin, Slice, Split, Sqrt, Tanh, Where	Add, Relu
One-to-Many	Elementwise w/ broadcast, Expand, Gather, Resize, Upsample	Expand
Many-to-Many	AveragePool, CONV, ConvTranspose, CumSum, Einsum, GEMM, InstanceNormalization, MaxPool, Reduce (e.g. ReduceProd, ReduceMean), Softmax	Conv, GEMM
Reorganize	Flatten, Reshape, Squeeze, Unsqueeze	Reshape
Shuffle	DepthToSpace, SpaceToDepth, Transpose	Transpose

execution framework. The optimized framework with operator fusion is called DNNFusion.

DNNFusion is extensively evaluated on 15 cutting-edge DNN models with 5 types of tasks, varied model sizes, and different layer counts on mobile devices. Comparing with four popular state-of-the-art end-to-end DNN execution frameworks, MNN [29], TVM [12], TensorFlow-Lite [1], and Pytorch-Mobile [48], DNNFusion achieves up to 8.8× more loop fusions, 9.3× speedup with our proposed advanced operator fusion. Particularly, DNNFusion *for the first time* allows many latest DNN models that are not supported by any existing end-to-end frameworks to run on mobile devices efficiently, even in real-time. Moreover, DNNFusion improves cache performance and device utilization – thus, enabling execution on devices with more restricted resources – and reduces performance tuning time during compilation.

2 Blessing and Curse of Deep Layers

This section presents a study that motivates our work, by demonstrating that it is challenging to execute deep(er) neural networks efficiently, particularly on resource-constraint mobile devices, due to the high memory and computation requirements.

As we stated earlier, there has been a trend towards deeper DNNs. With increasing amount of computation, there has also been a trend towards reducing the computation by reducing the weight size. Consider the well-known Natural Language Processing (NLP) model, BERT [19] as an example. TFLite takes 985ms to inference BERT on the latest CPU of Snapdragon 865. In recent efforts [55, 65] (MobileBERT, GPT-2), machine learning researchers have addressed this issue by reducing the weight size on each layer and thus training thinner and deeper models to balance the computation workload and model accuracy.

However, we have observed that the depth of the model is the critical impediment to efficient execution. Our experimental study has correlated execution efficiency with the total amount of computation and the number of layers (Table 1). Particularly, we can see that although DistilBERT [60] and VGG-16 [62] have a similar number of computations (while having 457 and 51 layers, respectively), DistilBERT's execution performance (78 GFLOPs/S) is much worse than VGG's (320 GFLOPs/S). This is mainly because of two reasons. First, models with more layers usually generate more intermediate results, thus increasing the memory/cache pressure. Second, deep models usually have an insufficient amount of computations in each layer, thus degrading the processor's utilization, particularly for GPUs. Operator fusion can be an effective technique to reduce memory requirements and improve efficiency, and is the focus of our study.

3 Classification of DNN Operators and Fusion Opportunity Analysis

This section establishes the basis for our approach, by classifying DNN operators and their combinations.

3.1 DNN Operators Classification

This work carefully studied all operators supported by a popular (and general) DNN ecosystem ONNX (Open Neural Network Exchange) [47], and finds that the mapping relation between (each) input and output of each operator is critical to determine both the profitability and correct implementation of fusion optimization. Moreover, it is possible for us to classify all operators into five high-level abstract types based on the relationship between input elements and output elements. These five types are One-to-One, One-to-Many, Many-to-Many (which includes Many-to-One, but we do not consider it separately here), Reorganize, and Shuffle. This classification serves as the foundation of our proposed fusion framework. Table 2 shows more details of this operator classification and gives one or two representative examples for each mapping type. If an operator has only one input or multiple inputs with the same mapping type to the output, the mapping type of this operator is decided by its any input/output pair. If multiple input/output pairs with varied mapping types exist, this operator's mapping type is decided by the more complex mapping type[1].

Assuming each input element can be denoted as $x[d_1, \ldots, d_n]$, where x means the operand of an operator and $d_1, \ldots, d_n$ denotes the index for an element of an operand, the mapping types between one input and one output are classified as follows.

[1]Order in increasing order of complexity: One-to-One, Reorganize, Shuffle, One-to-Many, and Many-to-Many

Table 3. Mapping type analysis. The first column and the first row (both without color) show the mapping types of first and second operators, respectively, before fusion, and the colored cells show the mapping type of the operator after fusion. Green implies that these fusion combinations can be fused directly (i.e., they are profitable). Red implies that these fusions are unprofitable. Yellow implies that further profiling is required to determine profitability.

First op \ Second op	One-to-One	One-to-Many	Many-to-Many	Reorganize	Shuffle
One-to-One	One-to-One	One-to-Many	Many-to-Many	Reorganize	Shuffle
One-to-Many	One-to-Many	One-to-Many	×	One-to-Many	One-to-Many
Many-to-Many	Many-to-Many	Many-to-Many	×	Many-to-Many	Many-to-Many
Reorganize	Reorganize	One-to-Many	Many-to-Many	Reorganize	Reorganize
Shuffle	Shuffle	One-to-Many	Many-to-Many	Reorganize	Shuffle

- **One-to-One:** There is a set of functions $F, f_1, \ldots, f_n$, such that
$$y[d_1, \ldots, d_n] = F(x[f_1(d_1,), \ldots, f_n(d_n)])$$
and there is a 1-1 mapping between each $[d_1, \ldots, d_n]$ and the corresponding $[f_1(d_1,) \ldots, f_n(d_n)]$ used to compute it.
- **One-to-Many:** There is a set of functions $F, f_1, \ldots, f_n$, such that:
$$y[e_1, \ldots, e_m] = F(x[f_1(d_1), \ldots, f_n(d_n)])$$
where $m > n$, and there is a One-to-Many relationship between $[f_1(d_1), \ldots, f_n(d_n)]$ and $[e_1, \ldots, e_m]$.
- **Many-to-Many:** There is a set of functions $f_1^1, \ldots, f_n^1, \ldots f_1^k, \ldots, f_n^k$, such that:
$$y[e_1, \ldots, e_m] = F(x^1[f_1^1(d_1), \ldots, f_n^1(d_n)], \ldots, x^k[f_1^k(d_1), \ldots, f_k^k(d_n)]).$$
- **Reorganize:** We have
$$y[e_1, \ldots, e_m] = x[f_1(d_1), \ldots, f_n(d_n)]$$
and there is a 1-1 relationship between each $[e_1, \ldots, e_m]$ and the corresponding $[f_1(d_1), \ldots, f_n(d_n)]$.
- **Shuffle:** There is a set of functions $F, f_1, \ldots, f_n$, where F is a *permutation* function, such that,
$$y[e_1, \ldots, e_n] = x[f_1(d_{F(1)}), \ldots, f_n(d_{F(n)})].$$

3.2 Fusion Opportunity Analysis

Based on the mapping type of each operator, this work proposes a new fusion analysis. The basic idea is that given two fusion candidate operators with a certain combination of mapping types, it is possible to: 1) infer the mapping type of the resulting fused operation; and 2) simplify the profitability evaluation and correct implementation of this fusion.

Table 3 shows the details of this analysis. The first column and the first row (without any color) show the mapping types of the first and the second operator to be fused and the colored cells show the mapping type of the resulting operator. It further classifies the fusion of this combination of mapping types into three groups (shown as green, yellow, and red, respectively). Green implies that these fusions are legal and profitable and no further analysis is required. Red implies that these fusions are known to be either illegal or clearly not profitable. Yellow implies that these fusions are legal; however, further profiling is required to determine profitability. This analysis eliminates the need for anytime runtime analysis or autotuning for red and green cases. For remaining (yellow) cases, we can further accelerate compilation using a *profiling database* that stores the execution results of various fusion combinations collected offline.

These five mapping types have a range of what we call *transformation impedance* (which we informally define as a metric to qualitatively express the difficulty to fuse), i.e., when they are fused with another type, they have different capability of deciding the fused mapping type. One-to-One has the lowest transformation impedance among all five types, whereas Reorganize and Shuffle's transformation impedance is in the middle, i.e., they can transform One-to-One to their types while they cannot transform others. One-to-Many and Many-to-Many have the strongest transformation impedance, i.e., the resulted mapping type is decided by them solely when they are fused with other operators. Moreover, One-to-Many and Many-to-Many have the same capability, and Reorganize and Shuffle have the same as well.

We elaborate on the following representative combinations to provide intuition behind the Table 3.

- *One-to-One with others.* When a One-to-One operator (Op_1 with the input `I` and the output `O`) is fused with an operator of any type (Op_2), i.e., Op_2 takes `O` as the input, the memory access to each element of `O` can be mapped to the access to each element of `I`, as long as this mapping function is known. Unlike general programs where the dependencies can be more complex, the use of tensors and a limited set of operators limits the type of mappings, and DNN operators carry this mapping information. Our analysis leverages this high-level operator information to ensure the correctness of these fusions. Moreover, this fusion usually requires limited number of registers and does not incur extra overhead like data copying or redundant computations, so they are profitable. Take a case that fuses `Add` and `GEMM` in either order. Each element in the output of `Add` can be replaced by two elements in the two inputs of `Add`, ensuring correct and profitable fusion, irrespective of the order of these operations.
- *Reorder or Shuffle with others.* Both types are variants of One-to-One with a special mapping function between the input and the output. Above reasons for the correctness analysis are also applied here; however, when fusing with One-to-Many or Many-to-Many types operators, profitability needs to be validated with further profiling because of the possibility of introduced data copying, change in data access order, or redundant computations.

As an example, consider Expand and Transpose operators – Expand copies the input tensor with a continuous memory access pattern, whereas, Transpose transposes the input tensor to the output tensor according to the permutation in operator properties. Thus, the resulting fused operation may not have continuous memory accesses.

- *One-to-Many with Many-to-Many.* Take the case that Expand followed by Conv – as Conv reads the feature map input tensor with continuous access, while a One-to-Many operator can distribute the continuous input tensor elements. As it is very desirable for the (compute-intensive) Many-to-Many operators to read the input tensors in a continuous way, we consider this fusion unprofitable.
- *Many-to-Many with Many-to-Many.* When a Many-to-One mapping operator is followed by a Many-to-One operator, e.g. Conv followed by another Conv, attempting a combined execution will be too complicated and will likely negatively impact register and cache usage. Thus, we consider them unprofitable.
- *Many-to-Many with One-to-Many.* When a Many-to-One mapping operator is followed by a One-to-Many operator, e.g. Conv followed by Expand or Resize, a combined execution may or may not have a desirable data access pattern. When Conv is combined with Expand, as Expand operator only expands a single dimension of the input, so it will not adversely affect the computation pattern of Conv. On the other hand, if Conv is combined with a Resize that will copy the input tensor along different dimensions, it can negatively impact the computation of Conv. Thus, we consider such cases to be requiring further profiling.

Extended Computational Graph. Based on the analysis above and as a background for the methods we will present next, we introduce Extended Computational Graph (ECG) as our intermediate representation (IR). As the name suggests, this represents builds on top of the (traditional) Computational Graph [12], which captures the data-flow and basic operator information like the operator type and parameters. ECG contains more fusion-related information, including `mapping_type` indicating the mapping type of each operator, `IR_removable` denoting if an intermediate result can be removed completely (which is true only if all its successors can be fused and which is calculated during fusion), and mathematical properties of the operations like whether the associative, commutative, and/or distributed properties hold.

4 DNNFusion's Design

4.1 Overview of DNNFusion

Figure 1 shows an overview of DNNFusion. It takes the computational graph generated from compiler-based DNN execution frameworks (e.g., TVM [12], and MNN [29]) as the input, and adds key information to create the Extended Computational Graph (ECG). Based on this ECG, the main compiler optimization and code generation stage of DNNFusion consists of three components: ❶ mathematical-property-based graph rewriting (Section 4.2), ❷ lightweight profile-driven fusion plan exploration (Section 4.3), and ❸ fusion code generation and other advanced fusion-based optimizations (Section 4.4).

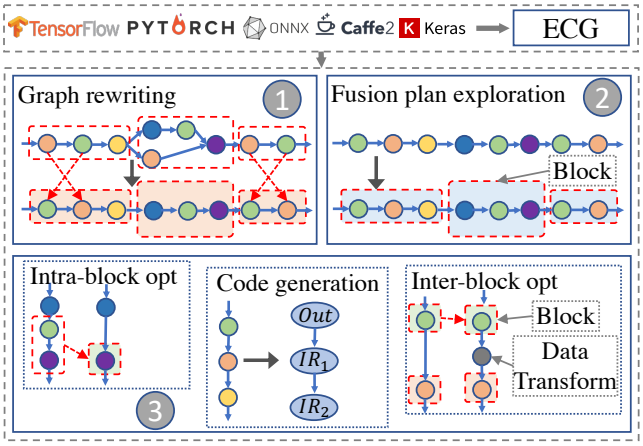


Figure 1. DNNFusion overview.

4.2 Mathematical-Property-Based Graph Rewriting

DNNFusion first employs a mathematical-property based graph rewriting pass to optimize the Extended Computational Graph (ECG). With this pass, DNNFusion is able to 1) remove unnecessary operations, 2) eliminate redundant intermediate data copies, and 3) replace costly (combination of) operators with more efficient ones. This graph rewriting carried out here is in the spirit of the classical compiler optimization of strength reduction [14]; however, here it is performed on complicated operators on matrices or tensors rather than on scalar expressions. Moreover, the rules we present are more complex and involved, and are based on operations that are common in DNNs. More importantly, compared to existing efforts on computational graph substitution (e.g., TASO [28]), our graph rewriting is designed to work in conjunction with operator fusion and identifies a set of operators and rules for that specific purpose. Our evaluation results (Section 5) show that with graph rewriting, there are 18% fewer fused layers left after fusion on GPT-2. We also do an experimental comparison against TASO later in this paper.

Figure 2 shows specific examples of leveraged mathematical properties (distributive, communicative, and associative). Table 4 shows a more complete set of rules. This table also shows the computation size (in #FLOPS) before and after the rewriting. Our rules mainly focus on operators in the One-to-One mapping type (e.g., element-wise multiplication, addition, reciprocal, square root, and others) and several reduction operators that are in Many-to-Many (e.g., ReduceSum and ReduceProd) – this is because these operators usually follow our defined mathematical properties. DNNFusion uses

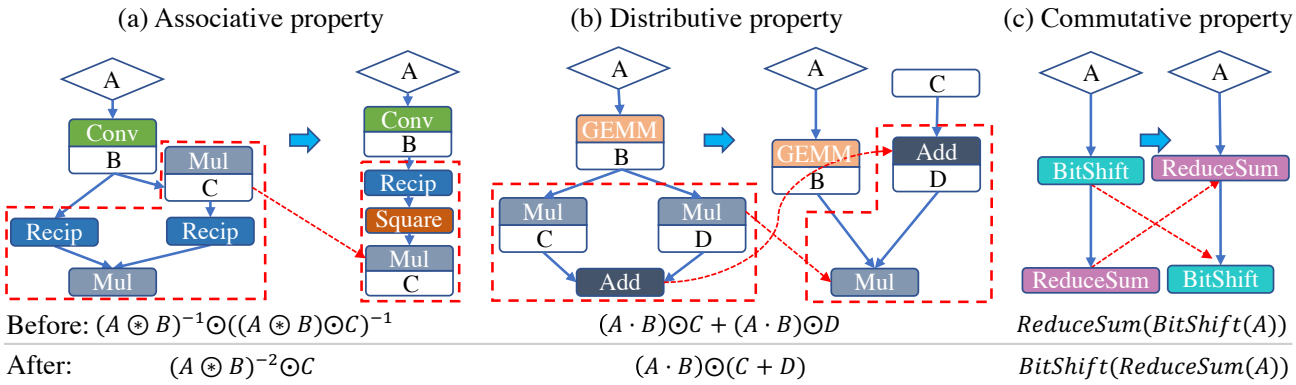


Figure 2. Examples of graph rewriting with mathematical properties. Associative property explores the optimal execution order of operators and replaces the expensive combination of operators with a cheaper one. Distributive property explores the common combination of operators and simplifies the computation structure. Commutative property switches the execution order of operators to reduce the overall computation. Note: the letter below each operator (e.g., `B` below `Conv` in (a)) or the letter in rectangle (e.g., `C` in (b)) denotes that this input is from model weights rather than an intermediate result. The letter in diamond (e.g., `A`) means that this is the input of this operator block, which could be the input of the model or intermediate result from a prior block. The intermediate results within this block are omitted for readability.

Table 4. Graph rewriting with mathematical properties. Only representative graph rewriting rules are listed due to space limitation. In summary, DNNFusion derives 45, 38, and 66 graph rewriting rules in the category of Associative, Distributive, and Communicative, respectively. We omit unrelated operators for better readability. $\odot, +, -, Abs, Recip, Square, \sqrt{}$ mean element-wise multiplication, addition, subtraction, absolute, reciprocal, square, and square root, respectively. `BitShift` calculates the bit shifted value of elements of a given tensor element-wisely. `ReduceSum` and `ReduceProd` calculate the reduced summation and production of elements of an input tensor along an axis. `Exp` calculates the exponent of elements in a given input tensor element-wisely. #FLOPS denotes the number of floating point operations

Property	Without graph rewriting		With graph rewriting	
	Graph structure in equation	#FLOPS	Graph structure in equation	#FLOPS
Associative	$Recip(A) \odot Recip(A \odot B)$	$4*m*n$	$Square(Recip(A)) \odot B$	$3*m*n$
	$(A \odot \sqrt{B}) \odot (\sqrt{B} \odot C)$	$5*m*n$	$A \odot B \odot C$	$2*m*n$
	$Abs(A) \odot B \odot Abs(C)$ †	$4*m*n$	$Abs(A \odot C) \odot B$	$3*m*n$
	$(A \odot ReduceSum(B)) \odot (ReduceSum(B) \odot C)$ ¶	$5*m*n$	$A \odot Square(ReduceSum(B)) \odot C$	$3*m*n+m$
Distributive	$A \odot C + A \odot B$	$3*m*n$	$(A+B) \odot C$	$2*m*n$
	$A + A \odot B$	$2*m*n$	$A \odot (B+1)$	$2*m*n$ §
	$Square(A+B) - (A+B) \odot C$	$5*m*n$	$(A+B) \odot (A+B-C)$	$3*m*n$
Commutative	$A \odot B$	$m*n$	$B \odot A$	$m*n$ ‡
	$ReduceSum(BitShift(A))$ ¶	$2*m*n$	$BitShift(ReduceSum(A))$	$m*n+m$
	$ReduceProd(Exp(A))$ ¶	$2*m*n$	$Exp(ReduceSum(A))$	$m*n+m$

§ Although #FLOPS is not reduced, A is loaded once instead of twice.

† First use commutative property to swap B and $Abs(C)$, then apply associative property.

‡ Even though this pattern has no #FLOPS gains, it can enable further optimization, e.g the case of †.

¶ #FLOPS is calculated by assuming the reduction of ReduceSum/ReduceProd is along with the inner-most dimension.

#FLOPs (rather than temporary output size or memory footprint) as the metric to drive graph rewriting mainly because of two reasons: first, in most of the applications scenarios of these rules, the temporary output size keeps the same before and after graph rewriting, and second, the size of the temporary output in a majority of other cases becomes a non-issue because fusion is applied after rewriting. For a small number of remaining cases, i.e., where temporary output size changes and the fusion is not applied, more sophisticated methods will be considered in the future.

We now elaborate on some of the rules presented in Table 4, which were also depicted in Figure 2.

- **Associative:** By leveraging the associative property, the graph rewriting pass can identify an optimized order of operators execution, and hence replace the expensive combination of operators with a new cheaper one. Figure 2 (a) shows an example, in which a combination of two `Recip`

operators and two Mul operators is replaced by a combination of a Recip, a Square, and a Mul. The latter is more efficient as it eliminates a Mul operator and the intermediate result size is significantly reduced, leading to reduced register pressure after subsequent fusion.

- **Distributive:** Following the same ideas as above, applying distributive property also enables optimization opportunities. As shown in Figure 2 (b), the combination of two Mul operators and an Add can be replaced by an Add followed by a Mul, thus eliminating an unnecessary operator.
- **Commutative:** The property guaranties the legality of swapping the position of two operators, which usually results in computation reduction. As shown in Figure 2 (c), BitShift [2] and ReduceSum[3] satisfy communicative property, thus ReduceSum can be scheduled to execute before BitShift, reducing the number of elements on which BitShift is applied.

DNNFusion employs pattern matching [36, 37] to recognize rewriting candidates. However, associative and commutative matching is NP-complete [5]. Therefore, DNNFusion first partitions the entire Extended Computational Graph into many sub-graphs by considering operators with neither of associative, communicative, or distributive properties as partitioning points within the original graph. Within each sub-graph, DNNFusion can explore all possible patterns and pattern combinations because these sub-graphs have limited number of operators. More specifically, all matching rules within a partition are considered and the rule leading to the largest reduction in #FLOPS is applied. This process is repeated till there are no additional matching rules within the partition. DNNFusion chooses this greedy scheme to keep the optimization overheads low.

4.3 Light-Weight Profile-Driven Fusion Plan Exploration

4.3.1 Overall Idea. Optimal fusion plan generation requires a large search space [8, 22] and has been shown to be NP-complete [15, 32]. To keep the process at manageable costs, DNNFusion explores fusion plans by employing a new light-weight (greedy) approach based on our proposed Extended Computational Graph (ECG) IR and our classification of operations into mapping types.

The high-level ideas are as follows. First, DNNFusion selects the starting operators (called *fusion seed operators*) from our ECG to restrict the search space. This is based on a key insight that operators of One-to-One mapping type have the potential to yield more benefits because they a) potentially result in fusion of more layers, including both with their predecessors and successors because of what we refer to as lower transformation impedance, and b) have lower memory requirements and need for fewer registers among

[2]Calculate the bit shifted value of elements of a given tensor element-wisely.
[3]Calculate the reduced sum of elements of an input tensor along an axis.

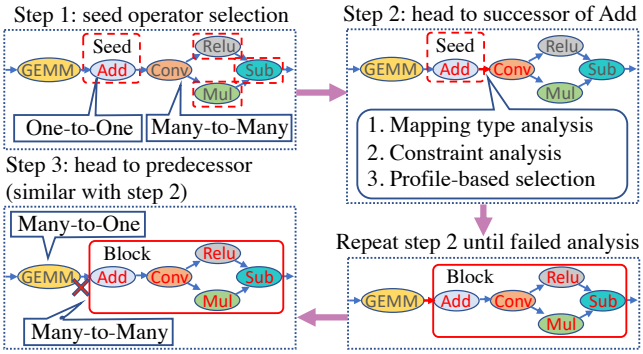


Figure 3. An example of fusion plan exploration. Assume Add, Conv, Relu, Mul, and Sub have identical output shape and IRS size.

all mapping types. Second, starting with these seed operators, DNNFusion explores fusion opportunities along the seed operator's successors and predecessors, respectively. Third, DNNFusion creates fusion plans based on an approach that combines machine-independent mapping type analysis and a *profiling result database*. The mapping type analysis follows Table 3 to check the operators' mapping type combination (in ECG) to decide if these operators should be fused. Such mapping eliminates unnecessary profile data lookup for most cases.

4.3.2 Fusion Plan Generation Algorithm. List 1 shows our detailed fusion plan exploration algorithm. Its goal is to generate candidate fusion blocks that are further optimized by subsequent intra-block optimizations (Section 4.4) before fusion code generation. Figure 3 illustrates its basic idea with a simplified example. This algorithm consists of three main steps:

Step I: Fusion seed operator(s) selection. DNNFusion selects the One-to-One operator with the minimum intermediate result as the fusion seed operator (as shown in Listing 1 lines 1 to 5). This heuristic is used because a smaller intermediate result makes fusion more profitable. This may seem counter-intuitive because fusing the operators with larger intermediate results usually results in more benefits. However, DNNFusion has a different goal, i.e., to ultimately enable more fusions to occur. Starting with a pair of operators with smaller intermediate results creates opportunities to fuse more (smaller) operators together, increase overall computation granularity, and hence enable higher parallelism and better load balance for the entire DNN computation. If multiple seed operators with the same minimum size of intermediate results exist, DNNFusion initiates fusion with them one after another (unless another seed is grouped to the same candidate fusion block). In Figure 3, Add, Relu, Mul, and Sub are in One-to-One type (with an identical intermediate result size), then Add is selected as the seed for the first round of fusion plan exploration.

```
1  def generate_seed(ops):
2    # find all one_to_one mapping operators
3    oto_ops = find_all_one_to_one(ops)
4    # find the operator with minimum IRS size
5    return min(op.IRS_size for op in oto_ops)
6
7  def fuse_successor(op, successor, block):
8    # Step 2.1: check the mapping relationship
9    relation = mapping_check(op, successor)
10   # return if successor can not be fused
11   if relation == fuse_break: return
12   # Step 2.2: check the constraint requirement
13   if not check_constraint(op, successor, block):
14     return
15   # fuse by profile-based selection
16   if relation == fuse_depend:
17     # Step 2.3: get latency w/ database/runtime
18     temp_latency = latency(block + successor)
19     if temp_latency > latency(block, successor):
20         return
21   block = op + successor
22   # Step 2.4: recursively head to successor
23   for fusing_op in successors(successor):
24     fuse_successor(successor, fusing_op, block)
25
26 # Similar with fuse_successor
27 def fuse_predecessor(op, predecessor, block):
28   # Similar with step 2.1, 2.2, 2.3, 2.4
29
30 # <Algorithm Entry>
31 unfused_ops = all_operators
32 # Step 1: start fuse from the selected seed
33 while(sp = generate_seed(unfused_ops))
34   block = [sp]
35   # Step 2: head to successor
36   for successor in successors(sp):
37     fuse_successor(sp, successor, block)
38   # Step 3: head to predecessor
39   for predecessor in predecessors(sp):
40     fuse_predecessor(sp, predecessor, block)
41   unfused_ops = unfused_ops - block
```

Listing 1. Fusion plan generation

Step II: Propagated exploration along seed's successors. Each operator may have one or multiple immediate predecessors and successors. DNNFusion first processes the seed operator's successors one by one (Listing 1 Lines 7 to 24). At any stage in this recursive exploration, if a node cannot be fused with any of its immediate successors, fusion is not considered any further. Broadly, this step proceeds as follows. First, *mapping type analysis* (Listing 1 Step 2.1) categorizes the potential fusion result into three types based on Table 3: 1) `fuse_break` indicates this is a Red case, and fusion should be aborted; 2) `fuse_through` indicates that this is a Green case, and should be proceeded without any further analysis; 3) `fuse_depend` indicates that this is a Yellow case, requiring a profile data lookup. Second, a *constraints check* (Listing 1 Step 2.2) is applied to analyze if further fusion is likely undesirable, i.e, it incurs too many overheads (e.g., can cause excessive register spills). Using an empirically determined

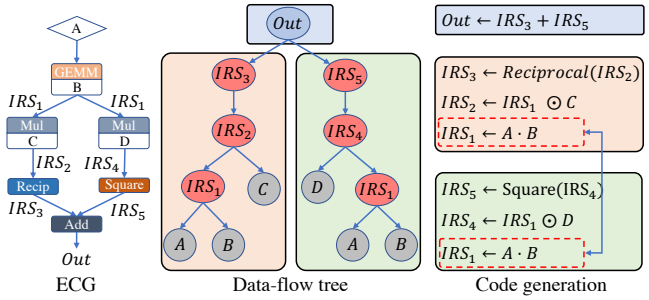


Figure 4. Code generation.

threshold, the algorithm can decide to not consider any additional fusion candidates. Otherwise, DNNFusion continues exploring fusion candidates recursively. Figure 3 shows an example of fusing `Add` with `Conv` and other operators with this step. After this step, the generated candidate fusion block has a mapping type of Many-to-One, and includes five operators (`Add`, `Conv`, `Relu`, `Mul`, and `Sub`).

Step III: Propagated exploration along seed's predecessors. After processing along the seed's successor direction, DNNFusion processes along the seed's predecessors direction with the same algorithm as Step II (In fact, Step III and Step II can be swapped). However, one difference is that if an operator has multiple immediate predecessors, there is an option of fusing with some, but not all, of these immediate predecessors. In the example in Figure 3, the first attempt of fusing current candidate fusion block with `GEMM` fails because both of them are of many-to-one mapping type. Table 3 indicates this is a `fuse_break` case, so `GEMM` is not included in this candidate fusion block.

Iterate. DNNFusion completes a round of fusion plan generation with above steps. If more fusion seeds exist, it will iterate from Step II with the next seed until no additional fusion seed is available.

4.4 Fusion Code Generation and Optimizations

4.4.1 Fusion Code Generation. Once fusion blocks have been selected by our algorithm, DNNFusion generates fused code for each fusion block with a *data-flow tree* (DFT) built from the Extended Computational Graph (ECG) and a set of pre-defined code generation rules. DNNFusion generates C++ code for mobile CPU and OpenCL for mobile GPU, respectively. More specifically, DNNFusion traverses DFT and generates fused code for each pair of operators to be fused by leveraging the code generation rules that are based on abstract mapping types (e.g., One-to-One). 23 code generation rules are defined for each of mobile CPU and mobile GPU, with one rule corresponding to a green or yellow cell in Table 3. The basic idea is that as long as the operators are of the same type, the same rules lead to efficient code. While fusing more than two operators, these rules are invoked each time two operators are fused. Finally, the subsequent code optimizations (e.g, vectorization, unrolling, tiling, and

memory/register optimizations, and auto-tuning of these optimizations) are handled by our existing framework called PatDNN [46], thus not a major contribution of this paper. Note that almost each fusion generates a new operator (and its codes) that is not present in the original operator library; however, once the new operator (and its code) is generated, it can be used for both the current model and future models.

Figure 4 shows an example of the code generation. To elaborate, DNNFusion first generates a data-flow tree (DFT) from the Extended Computational Graph (ECG). This DFT represents the final output (`Out`), all intermediate results (`IRS`), and all inputs (`A`, `B`, `C`, and `D`) with the edges reversed as compared to the ECG (i.e., the parent node depends on the children nodes). During the fused code generation, DNNFusion traverses this DFT to recognize the input/output data dependence (and fuses corresponding ECG operations), recursively. The right-hand side of Figure 4, shows an example of this DFT traversal (the fused code generation based on the pre-defined code generation rules is omitted in this Figure for readability and is introduced in the next paragraph). First, DNNFusion recognizes that *Out* depends on IRS_3 + IRS_5; next, it recognizes that IRS_3 depends on reciprocal of IRS_2, and so on, until reaching the input of A, B, C, D. It is worth noting DNNFusion can also find *redundant computations* in DFT with a common sub-tree identification and eliminate them during code generation. In our example, both `Mul` operators use IRS_1, resulting in a common sub-tree in DFT, so the recognition in two red boxes of Figure 4 is only taken once.

During this DFT traversal, DNNFusion employs the pre-defined code generation rules to generate the code for each pair of operators to be fused. For the example shown in Figure 4, DNNFusion first fuses `Add` with its left input branch `Recip`. Both `Add` and `Recip` belong to One-to-One mapping, and hence the fused operator is also One-to-One. DNNFusion keeps fusing `Mul` (One-to-One) with this newly fused operator, and the result is still One-to-One. Next, this newly generated operator is fused with `GEMM` (Many-to-One), generating a new Many-to-One operator. Similar steps are taken along the right input branch of `Add` until all operators are fused into a single new Many-to-One operator. DNNFusion relies on the DFT traversal introduced in the prior paragraph to figure out the input/output data dependence, and employs the operator mapping type to handle the index mapping relationship and generate proper nested loop structures.

To explain this further, here is an example with more complicated mapping types: `GEMM` (Many-to-Many) + `Div` (One-to-One) + `Transpose` (Shuffle). First, DNNFusion fuses `Transpose` and `Div`, a case of ("Shuffle + One-to-One") by first permuting the loop in the `Transpose` operator and then fusing it with the `Div` operator. It generates a new operator of the type Shuffle. Next, DNNFusion fuses GEMM (Many-to-Many type) with this new operator (Shuffle type), in which DNNFusion maps output elements of GEMM to the destination that is decided by this new operator.

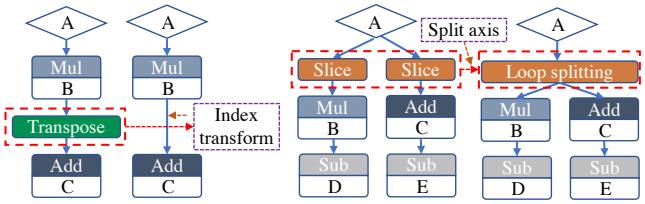


Figure 5. Data movement operators optimization.

4.4.2 Other Fusion-related Optimizations.

DNNFusion also includes several advanced optimizations enabled by our fusion analysis and fused code generation. They broadly can be characterized into two groups, *intra-fusion-block optimizations* that are performed on Extended Computational Graph (ECG) immediately before the code generation and *inter-fusion-block optimizations* on the generated fused code.

Intra-block Optimizations: Operators in Shuffle and Reorganize mapping types usually involve intensive data movement. We observed many of these time/memory consuming data operations can be eliminated. In particular, consider the case when the transformed data is used by only one subsequent operator because the data locality improvement brought this data transformation cannot be compensated by the overhead of intermediate results generation and storage. Figure 5 shows such examples – particularly, in these, data transpose and data slicing operations bring more overheads than the benefit. Thus, in such cases, DNNFusion replaces them with operations that have a changed data index. These optimizations are performed after graph rewriting and result in an ECG that should have a more efficient implementation.

Inter-block Optimization: Different operators prefer different data formats. Without the proposed graph rewriting optimizations and operator fusion, normally such choices are made at the level of each individual operator – however, this can result in redundant or unnecessary transformations. In contrast, DNNFusion considers the data format choice at a global level, thus avoiding redundant or unnecessary transformations. Currently, DNNFusion employs a heuristic approach to optimize the data format, which is as follows. For a specific fusion block, it identifies one *dominant* operator whose performance is impacted the most by the choice of the layout (e.g., `CONV`, `GEMM`, and `Softmax` are most likely to such operators). The optimal layout for this operation is then used for the entire fusion block. This heuristic approach works based on a key observation that most other *non-dominant* operators can employ any layout/format without their performance being significantly affected. A potential future work will be to consider more sophisticated cost models, including balancing the cost of reformatting the data with reductions in execution because of the optimized layout.

Table 5. Fusion rate evaluation: computation layer count and intermediate result size for all evaluated DNNs. CIL (Compute-Intensive Layer): each input is used more than once, e.g. MatMul, CONV. MIL (Memory-Intensive Layer): each input is used only once, e.g. Activation. IRS: intermediate results. '-' means this framework does not support this model.

Model	Type	Task	Layer counts and IRS sizes before opt.				Layer counts and IRS sizes after opt.					
			#CIL	#MIL	#Total layer	IRS size	MNN	TVM	TFLite	Pytorch	**DNNF**	**IRS size**
EfficientNet-B0	2D CNN	Image classification	82	227	309	108MB	199	195	201	210	**97**	**26MB**
VGG-16	2D CNN	Image classification	16	35	51	161MB	22	22	22	22	**17**	**52MB**
MobileNetV1-SSD	2D CNN	Object detection	16	48	202	110MB	138	124	138	148	**71**	**37MB**
YOLO-V4	2D CNN	Object detection	106	292	398	329MB	198	192	198	232	**135**	**205MB**
C3D	3D CNN	Action recognition	11	16	27	195MB	27	27	-	27	**16**	**90MB**
S3D	3D CNN	Action recognition	77	195	272	996MB	-	-	-	272	**98**	**356MB**
U-Net	2D CNN	Image segmentation	44	248	292	312MB	241	232	234	-	**82**	**158MB**
Faster R-CNN	R-CNN	Image segmentation	177	3,463	3,640	914MB	-	-	-	-	**942**	**374MB**
Mask R-CNN	R-CNN	Image segmentation	187	3,812	3,999	1,524MB	-	-	-	-	**981**	**543MB**
TinyBERT	Transformer	NLP	37	329	366	183MB	-	304†	322	-	**74**	**55MB**
DistilBERT	Transformer	NLP	55	402	457	540MB	-	416†	431	-	**109**	**197MB**
ALBERT	Transformer	NLP	98	838	936	1,260MB	-	746†	855	-	**225**	**320MB**
$BERT_{BASE}$	Transformer	NLP	109	867	976	915MB	-	760†	873	-	**216**	**196MB**
MobileBERT	Transformer	NLP	434	1,953	2,387	744MB	-	1,678†	2,128	-	**510**	**255MB**
GPT-2	Transformer	NLP	84	2,449	2,533	1,389MB	-	2,047†	2,223	-	**254**	**356MB**

† TVM does not support this model on mobile. This layer count number is collected on a laptop platform for reference.

5 Evaluation

DNNFusion is implemented on top of an existing end-to-end DNN execution framework called PatDNN [46] that supports both dense and sparse DNN execution. It has been shown in our previous work that PatDNN [46] performs slightly better than TVM, MNN, and TFLITE even without our proposed operator fusion. For readability, we also call this optimized framework DNNFusion. Our evaluation has four objectives: 1) demonstrate that the proposed fusion framework (together with graph rewriting) is effective by showing how DNNFusion outperforms other state-of-the-art frameworks, and no-fusion and fixed-pattern fusion implementations on various DNN models; 2) validating DNNFusion's generality by showing its efficient execution on both CPU and GPU on a wide spectrum of DNNs (for 5 types of tasks, with varied sizes, and layer counts ranging from relatively shallow to extremely deep); 3) analyzing the impact of different compiler optimizations on both execution time and compilation time; and 4) demonstrating the effective portability of DNNFusion by evaluating it on three different mobile phones.

More specifically, DNNFusion (also called `DNNF` for short) is compared against four popular state-of-the-art end-to-end DNN execution frameworks: MNN [29], TVM [12], TensorFlow-Lite (`TFLite`) [1], and Pytorch-Mobile (`Pytorch`) [48]. Because certain extremely deep neural networks are not supported by any of these existing frameworks (or just supported by their mobile CPU implementation), we also set a baseline by turning off DNNFusion's all fusion related optimizations (called `OurB`, i.e., our baseline version without fusion) and implement a version that optimizes `OurB` with fixed-pattern fusion (using operator fusion described in TVM [12]) (called `OurB+`), and compare DNNFusion against them.

5.1 Evaluation Setup

Models and datasets. DNNFusion is evaluated on 15 mainstream DNN models. Table 5 characterizes them with a comparison of their targeted task, number of parameters, total number of layers, and number of floating point operations (FLOPS). Particularly, we have 1) two image classification 2D CNNs (EfficientNet-B0 [66] and VGG-16 [62]), 2) two object detection two-dimensional (2D) CNNs (MobileNetV1-SSD [42] and YOLO-V4 [7]), 3) two action recognition three-dimensional (3D) CNNs (C3D [69] and S3D [73]), 4) one image segmentation 2D CNN (U-Net [59]) and two image segmentation R-CNNs (Mask R-CNN [26] and FasterRCNN [57]), and 5) six natural language processing (NLP) models (TinyBERT [30], DistilBERT [60], ALBERT [39], $BERT_{BASE}$, MobileBERT [65], and GPT-2 [55]).

Because the choice of datasets has a negligible impact on the final inference latency or relative execution speeds (and also because of space limitations), we report results from one dataset for each model. EfficientNet-B0 and VGG-16 are trained on ImageNet dataset [18]; MobileNetV1-SSD and YOLO-V4 are trained on MS COCO [41]; C3D and S3D are trained on UCF-101 [63]; U-Net, Faster R-CNN, and Mask R-CNN are trained on PASCAL VOC 2007 [23]; TinyBERT, DistilBERT, ALBERT, $BERT_{base}$, MobileBERT, and GPT-2 are trained on BooksCorpus [19] and English Wikipedia [19]. Because the model accuracy is identical among all frameworks, and also because of space limitations, we only focus on execution times and do not report accuracy.

Table 6. Inference latency comparison: DNNFusion, MNN, TVM, TFlite, and PyTorch on mobile CPU and GPU. #FLOPS denotes the number of floating point operations. OurB is our baseline implementation by turning off all fusion optimizations and OurB+ is OurB with a fixed-pattern fusion as TVM. DNNF is short for DNNFusion, i.e., our optimized version. '-' denotes this framework does not support this execution.

Model	#Params	#FLOPS	MNN (ms)		TVM (ms)		TFLite (ms)		Pytorch (ms)		OurB (ms)		OurB+ (ms)		**DNNF (ms)**	
			CPU	GPU	CPU	GPU	CPU	GPU	CPU	GPU	CPU	GPU	CPU	GPU	**CPU**	**GPU**
EfficientNet-B0	5.3M	0.8B	41	26	56	27	52	30	76	-	54	35	38	24	**16**	**10**
VGG-16	138M	31.0B	242	109	260	127	245	102	273	-	251	121	231	97	**171**	**65**
MobileNetV1-SSD	9.5M	3.0B	67	43	74	52	87	68	92	-	79	56	61	39	**33**	**17**
YOLO-V4	64M	34.6B	501	290	549	350	560	288	872	-	633	390	426	257	**235**	**117**
C3D	78M	77.0B	867	-	1,487	-	-	-	2,541	-	880	551	802	488	**582**	**301**
S3D	8.0M	79.6B	-	-	-	-	-	-	6,612	-	1,409	972	1,279	705	**710**	**324**
U-Net	2.1M	15.0B	181	106	210	120	302	117	271	-	227	142	168	92	**99**	**52**
Faster R-CNN	41M	47.0B	-	-	-	-	-	-	-	-	2,325	3,054	1,462	1,974	**862**	**531**
Mask R-CNN	44M	184B	-	-	-	-	-	-	-	-	5,539	6,483	3,907	4,768	**2,471**	**1,680**
TinyBERT	15M	4.1B	-	-	-	-	97	-	-	-	114	89	92	65	**51**	**30**
DistilBERT	66M	35.5B	-	-	-	-	510	-	-	-	573	504	467	457	**224**	**148**
ALBERT	83M	65.7B	-	-	-	-	974	-	-	-	1,033	1,178	923	973	**386**	**312**
$BERT_{Base}$	108M	67.3B	-	-	-	-	985	-	-	-	1,086	1,204	948	1,012	**394**	**293**
MobileBERT	25M	17.6B	-	-	-	-	342	-	-	-	448	563	326	397	**170**	**102**
GPT-2	125M	69.1B	-	-	-	-	1,102	-	-	-	1,350	1,467	990	1,106	**394**	**292**

Evaluation environment. The evaluations are carried out on a Samsung Galaxy S20 cell phone that has Snapdragon 865 processor [54], which comprises an octa-cores Kryo 585 CPU and Qualcomm Adreno 650 GPU yielding high performance with good power efficiency. For demonstrating portability, we further use a Samsung Galaxy S10 with a Snapdragon 855 [53] (Qualcomm Kryo 485 Octa-core CPU and a Qualcomm Adreno 640 GPU), and an Honor Magic 2 with a Kirin 980 [27] (ARM Octa-core CPU and a Mali-G76 GPU). All executions used 8 threads on mobile CPUs, and similarly all pipelines on mobile GPUs. 16-bit and 32-bit floating points are used for all GPU runs and CPU runs, respectively. All experiments were run 100 times but as the variance was very small, we only report averages.

5.2 Overall Mobile Inference Evaluation

Our comparison includes both fusion rate[4] and execution latency.

Fusion rate. Table 5 shows detailed layer counts (including computation-intensive (CIL), memory-intensive (MIL), and all layers), and intermediate result sizes for models before fusion and after fusion with different frameworks. Note that DNNFusion is the only end-to-end framework that can support all of the target models on both mobile CPU and mobile GPU. In Table 5, "-" implies that this framework does not support this model. Certain extremely deep neural networks (e.g., Faster R-CNN and Masker R-CNN) are not supported by any other frameworks on mobile devices because these frameworks either lack the support of multiple key operators and/or limited optimization supported in them lead to a large model execution footprint. For transformer-based models, only TFLite can support execution on mobile CPU (without GPU support).

Table 5 shows that compared with the other frameworks, DNNFusion results in better fusion rates, with 1.3× to 2.9×, 1.3× to 8.1×, 1.3× to 8.8×, and 1.3× to 2.8× over MNN, TVM, TFLite, and Pytorch, respectively. Particularly, compared with original models, DNNFusion yields more benefits for R-CNN and Transformer-based models (3.9× to 10.0× fusion rate) than 2D/3D CNNs (1.7× to 3.6× fusion rate). This is because 2D/3D CNNs have higher fractions of computation-intensive layers that are in either One-to-Many or Many-to-Many types, while transformer-based models have more memory-intensive layers that are in One-to-One, Shuffle, or Reorganize categories. The latter offers more fusion opportunities according to our mapping type analysis (Table 3). Because of the same reason, 3D CNNs have the lowest fusion rate because they are more compute-intensive. Moreover, comparing to TVM (that performs the best among all other frameworks), DNNFusion particularly yields more benefits for transformer-based models. This is because these models have more types of operators, and TVM's fixed pattern-based fusion cannot capture fusion opportunities among many types of operators while DNNFusion can. This result demonstrates that DNNFusion has a better generality.

Execution latency. Table 6 shows the execution latency evaluation results. Comparing with MNN, TVM, TFLite, and Pytorch, with fusion optimization, DNNFusion achieves the speedup of 1.4× to 2.6×, 1.5× to 3.5×, 1.4× to 3.3×, and 1.6× to 9.3×, respectively, on the mobile CPU. Focusing on mobile GPU, improvements over MNN, TVM, and TFLite are 1.7× to 2.6×, 2.0× to 3.1×, 1.6× to 4.0×, respectively, whereas

[4]Fusion rate = original layer count/fused layer count.

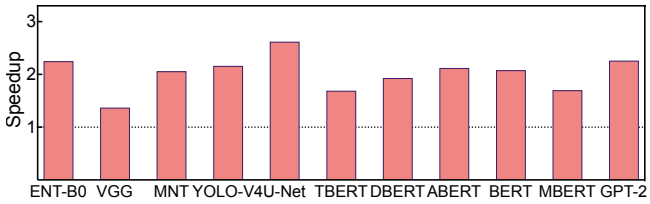
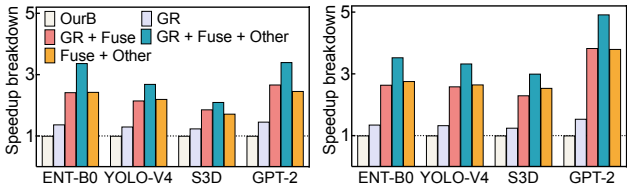


Figure 6. Speedup over TASO optimized execution on mobile CPU. The models (computational graphs) are optimized by TASO and then executed on TFLite.



(a) CPU. **(b)** GPU.

Figure 7. Optimization breakdown on y-axis: speedup over OurB, i.e. a version w/o fusion opt. GR, Fuse, and Other denote graph rewriting, fusion, and other fusion-related optimizations, respectively.

Pytorch does not support execution on this mobile GPU. The reason for speedups including the fact that our baseline implementation with a fixed-pattern fusion (`OurB+`) is already faster than other frameworks (with speedup of 1.04× to 5.2× on mobile CPU and 1.05× to 1.7× on mobile GPU), and with our more advanced fusion, DNNFusion achieves 1.4× to 2.5× speedup over `OurB+` on mobile CPU and 1.5× to 3.9× speedup on mobile GPU. In addition, comparing DNNFusion (with fusion) and our baseline without fusion (`OurB`), our advanced fusion brings 1.5× to 5.8× speedup. Moreover, fusion optimization offered by DNNFusion brings more benefits for mobile GPU than CPU, particularly for extremely deep models (e.g., Faster R-CNN and GPT-2). This is because mobile GPU offers more parallelism but smaller cache capacity compared to CPU, and GPU kernel launch incurs certain overheads, so it is more sensitive to intermediate data reduction, kernel launch reduction, and processor utilization increase that are brought by DNNFusion's fusion.

To further validate DNNFusion's performance, Figure 6 compares it with a state-of-the-art computational graph substitution approach mentioned earlier, TASO [28]. We use TASO to optimize all eleven models (computational graphs) supported by TFLite among those listed in Table 6, including EfficientNet-B0 (ENT-B0), VGG-16 (VGG), MobileNetV1-SSD (MNT), YOLO-V4, U-Net, TinyBERT (TBERT), DistilBERT (DBERT), ALBERT (ABERT), BERT_{Base} (BERT), MobileBERT (MBT), and GPT-2. Then, for our experiments, these models are executed under TFLite on mobile CPU (not GPU because TFLite lacks GPU support for many of these models). Compared with TASO, DNNFusion yields 1.4× to 2.6× speedup on mobile CPU. The graph rewriting in DNNFusion is designed to work in conjunction with operator fusion and identifies a set of operators and rules for that specific purpose, thus enabling more fusion opportunities. TASO does not emphasize the relationship between graph rewriting and fusion, resulting in less efficient execution as compared to DNNFusion.

5.3 Understanding Fusion Optimizations

This section studies the effect of our key optimizations.

Optimization breakdown. Figure 7 shows the impact of our proposed optimizations on latency with four models (EfficientNet-B0 (ENT-B0), YOLO-V4, S3D, and GPT-2) on both mobile CPU and GPU. Experiments on other models show a similar trend and are omitted due to space constraints. We evaluate each compiler-based optimization speedup incrementally over the `OurB` version. Compared with `OurB`, graph rewriting brings 1.2× to 1.5× speedup, fusion brings additional 1.6× to 2.2× speedup, and other optimizations (intra-block optimizations like data movement operator optimizations and inter-block optimizations like data format optimizations introduced in Section 4.4.2) bring additional 1.3× to 1.8× speedup on mobile CPU. On mobile GPU, these numbers are 1.3× to 1.5×, 2.1× to 3.3×, and 1.7× to 2.1×, respectively. Again, our fusion brings more benefit for mobile GPU than CPU due to the aforementioned reasons of memory, parallelism, and kernel launch overheads. Although graph rewriting by itself brings fewer benefits than the fusion, its hidden benefit is in enabling more fusion opportunities. Take GPT-2 as an example – without graph rewriting, the generated fused layer count is 310, while after rewriting, it is 254 (18% reduction). This is because graph rewriting simplified the computational graph structure. To further assess this impact of graph rewriting on operator fusion, the last bar (in orange) of Figure 7 reports the speedup of fusion with other optimizations only (i.e., without graph rewriting) over the baseline `OurB`. Compared with no graph rewriting, graph rewriting brings an additional 1.4× to 1.9× and 1.6× to 2.0× speedup (over `OurB`) on mobile CPU and GPU, respectively.

Memory and cache performance We compare memory (and cache) performance of DNNFusion with other frameworks on both mobile CPU and GPU. We only report YOLO-V4's results due to space constraint, and because it is one of the models supported by all frameworks. Figure 8 (a) left shows memory performance (measured in total memory accesses (MA) and memory consumption (MC)) – MA and MC are collected from Snapdragon Profiler [52], and Figure 8 (a) right shows cache miss count on data cache and TLB cache on mobile CPU. All values are normalized with respect to DNNFusion (i.e., our best version) for readability. DNNFusion outperforms other frameworks on both memory access count and memory consumption because it eliminates materialization of more intermediate results. Figure 8 (b) shows similar results on mobile GPU (excluding PyTorch because it does not support YOLO-V4 on mobile GPU). Mobile GPU results are generally better than CPU because mobile GPU

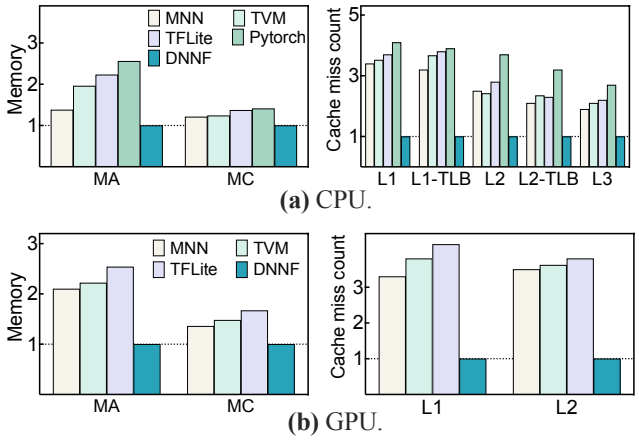


Figure 8. Memory (left) and cache miss (right) analysis. MA and MC denote memory access and memory consumption, respectively. Cache miss count is compared on L1/L2/L3 data cache and L1/L2 TLB cache on mobile CPU, and on L1/L2 data cache only on mobile GPU. All values are normalized w.r.t DNNF (the optimal version).

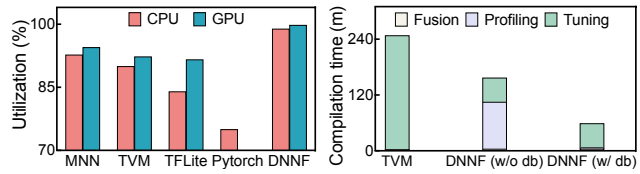


Figure 9. (a) Mobile CPU and GPU utilization. CPU utilization is averaged on 8-cores. **(b) Compilation time.** Comparison between TVM and DNNF for YOLO-V4 on mobile CPU. DNNF (w/o db) is without the presence of an existing profiling database; DNNF (w/ db) assumes such a database is pre-computed. `Fusion` is invisible as it spends very little time on both TVM and DNNF.

has smaller cache capacity and simpler hierarchy (GPU only has L1/L2 v.s., CPU has L1/L2/L3). Thus, intermediate results reduction leads to more gains on mobile GPU.

CPU/GPU utilization. Figure 9 (a) reports the mobile CPU and GPU utilization on YOLO-V4. This result is collected by Snapdragon Profiler [52]. It shows that DNNFusion results in the highest utilization on both CPU and GPU because its aggressive fusion groups more computation together, resulting in coarser-grained execution with less loop and parallel scheduling (and kernel launch for GPU only) overhead. Particularly, the GPU utilization is slightly higher than CPU because of its higher parallelism.

Compilation time. Figure 9 (b) compares the compilation time between TVM and DNNFusion for YOLO-V4 on mobile CPU. TVM's compilation time consists of operator fusion and other compiler optimizations (`Fusion`), and tuning for performance-critical parameters, e.g., tiling size, unrolling factors, and others (`Tuning`). `Tuning` dominates its

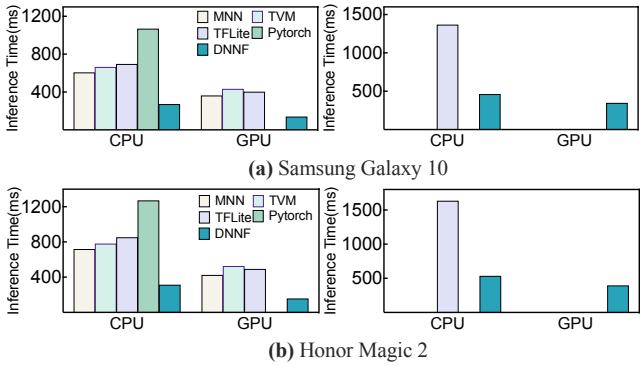


Figure 10. Portability evaluation. It is on Samsung Galaxy S10 and Honor Magic 2. Left two figures are YOLO-V4 and right two are GPT-2. Only TFLite supports GPT-2 on mobile CPU (no mobile GPU support).

compilation time, lasting for around 4 hours for YOLO-V4 on mobile CPU. DNNFusion's compilation time consists of operator fusion and other compiler optimizations (`Fusion`), profiling to analyze the fusion benefits (`Profiling`), and performance tuning (`Tuning`). DNNFusion's `Tuning` relies on an auto-tuner based on Genetic Algorithm (reported in our previous publication [46]) to generate the exploration space. Compared with AutoTVM [13], our auto-tuning's Genetic Algorithm allows parameter search to start with initializing an arbitrary number of chromosomes. Without our pre-existing profiling database, `Profiling` and `Tuning` dominate the compilation, requiring around 3 hours. With a pre-computed database, `Profiling` becomes very fast, and only `Tuning` dominates the compilation, requiring around 1 hour. After evaluating 15 models in Table 5, the profiling database consists of around 22K profiling entries with each one including operators' information (e.g., operator types, shape, and their combinations) and the latency achieved.

5.4 Portability

Figure 10 shows the execution latency on additional cell phones (Samsung Galaxy S10 and Honor Magic 2) to demonstrate effective portability. Only YOLO-V4 and GPT-2 are reported due to limited space. Other models show similar trends. In particular, DNNFusion shows a more stable performance on older generations of mobile devices. This is because our fusion significantly reduces the overall number of layers and intermediate result size, and older cell phones with more restricted resources are more sensitive to these.

6 Related Work

Operator fusion in end-to-end mobile DNN frameworks. Operator fusion is an important optimization in many state-of-the-art end-to-end mobile DNN execution frameworks that are based on computational graphs, such as MNN [29], TVM [12], TensorFlow-Lite [1], and Pytorch [48]. However, they all employ fixed-pattern fusion that is too restricted

to cover diverse operators and layer connections in deep models like BERT – for example, `ConvTranspose` + `ReLU` + `Concat` cannot be recognized in TVM as it is not one of the specified patterns. Other examples that can be handled by DNNFusion and cannot be recognized by TVM include `MatMul` + `Reshape` + `Transpose` + `Add` in GPT-2, and `Sub` + `Pow` + `ReduceMean` + `Add` + `Sqrt` in TinyBERT. Comparing to these frameworks, DNNFusion works by classifying both the operators and their combinations, thus enabling a much larger set of optimizations.

Operator fusion on other ML frameworks. There are certain recent frameworks that rely on polyhedral analysis to optimize DNN computations and support operator fusion. R-Stream·TF [51] shows a proof-of-concept adaptation of the R-Stream polyhedral compiler to TensorFlow. Tensor Comprehensions [70] is an end-to-end compilation framework built on a domain-specific polyhedral analysis. These frameworks do not support mobile execution (i.e. ARM architecture), and thus we cannot perform a direct comparison between DNNFusion and them. As we have stated earlier, DNNFusion maintains an operator view but builds a higher-level abstraction on them. In the future, we can combine DNNFusion's high-level abstraction to existing domain-specific polyhedral analysis. Similarly, another promising direction will be to integrate DNNFusion into other compilation-based DNN frameworks [25, 45] or other popular general tensor/matrix/-linear algebra computation frameworks, such as MLIR [40], Tiramisu [4], TACO [33, 34], Halide [56], and LGen [38, 64].

There also exist several other frameworks to optimize machine learning with operator fusion or fusion-based ideas. Closely related to DNNFusion– Rammer [43] relies on fix-pattern operator fusion to further reduce kernel launch overhead of their optimized scheduling, Cortex [24] proposes a set of optimizations based on kernel fusion for dynamic recursive models, TensorFlow XLA [67] offers a more general fusion method than fix-pattern operator fusion by supporting reduce operations and element-wise operations, and TensorFlow Grapper [68] provides an arithmetic optimizer that performs rewrites to achieve both fusion and arithmetic expression simplification (e.g., $a \times b + a \times c = a \times (b + c)$). Comparing with these frameworks, DNNFusion works by classifying the operators and their combinations into several mapping categories, thus resulting in a more aggressive fusion plan and more performance gains. Elgamal [22] and Boehm [8] presently optimize general machine learning algorithms (e.g., SVM and Kmeans) with operator fusion. These efforts have both different targets and techniques compared to DNNFusion.

Polyhedral-based and other loop fusion methods. Polyhedral analysis [10, 11, 20, 35, 49, 74] is a prominent approach that offers a general and rigorous foundation for loop transformation and optimization.

Many existing efforts [2, 3, 9] rely on a general polyhedral analysis to achieve optimized loop fusion. Pouchet *et al.* [50] have demonstrated that polyhedral analysis can decompose the loop optimization problem into sub-problems that have much lower complexity, enabling optimal selection. The problem arising because of a large number of operators in our target applications (models) is quite different, and thus there does not seem to be a direct application of Pouchet *et al.*'s approach in our context. There have also been other loop fusion efforts targeting general programs [17, 31, 32, 44]. In contrast to these general efforts, DNNFusion is more domain-specific, leveraging the knowledge of DNN computations with a higher-level abstraction to explore more aggressive loop fusion opportunities.

7 Conclusions and Future Work

This paper has presented a new loop fusion framework called DNNFusion. The key advantages of DNNFusion include: 1) a new high-level abstraction comprising mapping type of operators and their combinations and the Extended Computational Graph, and analyses on these abstractions, 2) a novel mathematical-property-based graph rewriting, and 3) an integrated fusion plan generation. DNNFusion is extensively evaluated on 15 diverse DNN models on multiple mobile devices, and evaluation results show that it outperforms four state-of-the-art DNN execution frameworks by up to 8.8× speedup, and *for the first time* allows many cutting-edge DNN models not supported by prior end-to-end frameworks to execute on mobile devices efficiently (even in real-time). In addition, DNNFusion improves both cache performance and device utilization, enabling execution on devices with more restricted resources. It also reduces performance tuning time during compilation.

Our future work will enhance DNNFusion by combining it with the latest model pruning advances [21, 46]. Though model pruning is effective, with fusion the dense versions are outperforming these efforts by having fewer layers. Thus, there is an opportunity to combine the two set of approaches to achieve an even better performance.

Acknowledgments

The authors would like to thank the anonymous reviewers for their valuable and thorough comments. The authors are especially grateful to the shepherd Tatiana Shpeisman for her innumerable helpful suggestions and comments that help improve this paper substantially. This work is supported in part by Jeffress Trust Awards in Interdisciplinary Research, and National Science Foundation (NSF) under CCF-1629392, CCF-2007793, CCF-1919117, and OAC-2034850. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of Thomas F. and Kate Miller Jeffress Memorial Trust, or NSF.

References

[1] Martin Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. 2016. TensorFlow: A system for large-scale machine learning. In *OSDI 2016*. 265–283.

[2] Aravind Acharya, Uday Bondhugula, and Albert Cohen. 2018. Polyhedral Auto-Transformation with No Integer Linear Programming. In *PLDI 2018* (Philadelphia, PA, USA). Association for Computing Machinery, New York, NY, USA, 529–542.

[3] Aravind Acharya, Uday Bondhugula, and Albert Cohen. 2020. Effective Loop Fusion in Polyhedral Compilation Using Fusion Conflict Graphs. *ACM Transactions on Architecture and Code Optimization (TACO)* 17, 4 (2020), 1–26. https://doi.org/10.1145/3416510

[4] Riyadh Baghdadi, Jessica Ray, Malek Ben Romdhane, Emanuele Del Sozzo, Abdurrahman Akkas, Yunming Zhang, Patricia Suriana, Shoaib Kamil, and Saman Amarasinghe. 2019. Tiramisu: A polyhedral compiler for expressing fast and portable code. In *CGO 2019*. 193–205.

[5] Dan Benanav, Deepak Kapur, and Paliath Narendran. 1987. Complexity of matching problems. *Journal of symbolic computation* 3, 1-2 (1987), 203–216.

[6] Sourav Bhattacharya and Nicholas D Lane. 2016. From smart to deep: Robust activity recognition on smartwatches using deep learning. In *2016 IEEE International Conference on Pervasive Computing and Communication Workshops (PerCom Workshops)*. IEEE, 1–6.

[7] Alexey Bochkovskiy, Chien-Yao Wang, and Hong-Yuan Mark Liao. 2020. YOLOv4: Optimal Speed and Accuracy of Object Detection. *arXiv preprint arXiv:2004.10934* (2020).

[8] Matthias Boehm, Berthold Reinwald, Dylan Hutchison, Prithviraj Sen, Alexandre V. Evfimievski, and Niketan Pansare. 2018. On Optimizing Operator Fusion Plans for Large-Scale Machine Learning in SystemML. *Proc. VLDB Endow.* 11, 12 (Aug. 2018), 1755–1768.

[9] Uday Bondhugula, Oktay Gunluk, Sanjeeb Dash, and Lakshminarayanan Renganarayanan. 2010. A Model for Fusion and Code Motion in an Automatic Parallelizing Compiler. In *PACT 2010* (Vienna, Austria). ACM, 343–352. https://doi.org/10.1145/1854273.1854317

[10] Uday Bondhugula, Albert Hartono, Jagannathan Ramanujam, and Ponnuswamy Sadayappan. 2008. A practical automatic polyhedral parallelizer and locality optimizer. In *PLDI 2008*. 101–113.

[11] Prasanth Chatarasi, Jun Shirako, and Vivek Sarkar. 2015. Polyhedral optimizations of explicitly parallel programs. In *PACT 2015*. IEEE, 213–226. https://doi.org/10.1109/PACT.2015.44

[12] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: An automated end-to-end optimizing compiler for deep learning. In *OSDI 2018*. 578–594.

[13] Tianqi Chen, Lianmin Zheng, Eddie Yan, Ziheng Jiang, Thierry Moreau, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. Learning to optimize tensor programs. *arXiv preprint arXiv:1805.08166* (2018).

[14] Keith D Cooper, L Taylor Simpson, and Christopher A Vick. 2001. Operator strength reduction. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 23, 5 (2001), 603–625.

[15] Alain Darte. 1999. On the complexity of loop fusion. In *1999 International Conference on Parallel Architectures and Compilation Techniques (Cat. No. PR00425)*. IEEE, 149–157.

[16] Jeffrey Dean, Greg Corrado, Rajat Monga, Kai Chen, Matthieu Devin, Mark Mao, Marc'aurelio Ranzato, Andrew Senior, Paul Tucker, Ke Yang, et al. 2012. Large scale distributed deep networks. In *Advances in neural information processing systems*. 1223–1231.

[17] Saumya K Debray. 1988. Unfold/fold transformations and loop optimization of logic programs. In *PLDI 1988*. 297–307.

[18] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. 2009. Imagenet: A large-scale hierarchical image database. In *CVPR 2009*. 248–255.

[19] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).

[20] Johannes Doerfert, Kevin Streit, Sebastian Hack, and Zino Benaissa. 2015. Polly's polyhedral scheduling in the presence of reductions. *arXiv preprint arXiv:1505.07716* (2015).

[21] Peiyan Dong, Siyue Wang, Wei Niu, Chengming Zhang, Sheng Lin, Zhengang Li, Yifan Gong, Bin Ren, Xue Lin, and Dingwen Tao. 2020. RTMobile: Beyond Real-Time Mobile Acceleration of RNNs for Speech Recognition. In *57th ACM/IEEE Design Automation Conference*. IEEE, 1–6.

[22] Tarek Elgamal, Shangyu Luo, Matthias Boehm, Alexandre V Evfimievski, Shirish Tatikonda, Berthold Reinwald, and Prithviraj Sen. 2017. SPOOF: Sum-Product Optimization and Operator Fusion for Large-Scale Machine Learning. In *CIDR*.

[23] Mark Everingham, Luc Van Gool, Christopher KI Williams, John Winn, and Andrew Zisserman. 2007. The PASCAL visual object classes challenge 2007 (VOC2007) results. (2007).

[24] Pratik Fegade, Tianqi Chen, Phil Gibbons, and Todd Mowry. 2020. Cortex: A Compiler for Recursive Deep Learning Models. *arXiv preprint arXiv:2011.01383* (2020).

[25] Sridhar Gopinath, Nikhil Ghanathe, Vivek Seshadri, and Rahul Sharma. 2019. Compiling KB-sized machine learning models to tiny IoT devices. In *PLDI 2019*. 79–95. https://doi.org/10.1145/3314221.3314597

[26] Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross Girshick. 2017. Mask r-cnn. In *ICCV 2017*. 2961–2969.

[27] Huawei. 2018. Kirin 980. https://consumer.huawei.com/en/campaign/kirin980

[28] Zhihao Jia, Oded Padon, James Thomas, Todd Warszawski, Matei Zaharia, and Alex Aiken. 2019. TASO: optimizing deep learning computation with automatic generation of graph substitutions. In *SOSP 2019*. 47–62. https://doi.org/10.1145/3341301.3359630

[29] Xiaotang Jiang, Huan Wang, Yiliu Chen, Ziqi Wu, Lichuan Wang, Bin Zou, Yafeng Yang, Zongyang Cui, Yu Cai, Tianhang Yu, Chengfei Lyu, and Zhihua Wu. 2020. MNN: A Universal and Efficient Inference Engine. In *Proceedings of Machine Learning and Systems*, I. Dhillon, D. Papailiopoulos, and V. Sze (Eds.). Vol. 2. 1–13.

[30] Xiaoqi Jiao, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, Fang Wang, and Qun Liu. 2019. Tinybert: Distilling bert for natural language understanding. *arXiv preprint arXiv:1909.10351* (2019).

[31] Mahmut Kandemir, A Choudhary, J Ramanujam, and Prithviraj Banerjee. 1998. Improving locality using loop and data transformations in an integrated framework. In *Proceedings. 31st Annual ACM/IEEE International Symposium on Microarchitecture*. IEEE, 285–296.

[32] Ken Kennedy and Kathryn S McKinley. 1993. Maximizing loop parallelism and improving data locality via loop fusion and distribution. In *International Workshop on Languages and Compilers for Parallel Computing*. Springer, 301–320.

[33] Fredrik Kjolstad, Peter Ahrens, Shoaib Kamil, and Saman Amarasinghe. 2019. Tensor algebra compilation with workspaces. In *CGO 2019*. IEEE, 180–192. https://doi.org/10.1109/CGO.2019.8661185

[34] Fredrik Kjolstad, Shoaib Kamil, Stephen Chou, David Lugato, and Saman Amarasinghe. 2017. The tensor algebra compiler. *Proceedings of the ACM on Programming Languages* 1, OOPSLA (2017), 1–29.

[35] Martin Kong, Richard Veras, Kevin Stock, Franz Franchetti, Louis-Noël Pouchet, and Ponnuswamy Sadayappan. 2013. When polyhedral transformations meet SIMD code generation. In *PLDI 2013*. 127–138.

[36] Emmanuel Kounalis and Denis Lugiez. 1991. Compilation of pattern matching with associative-commutative functions. In *Colloquium on Trees in Algebra and Programming*. Springer, 57–73.

[37] Manuel Krebber. 2017. Non-linear associative-commutative many-to-one pattern matching with sequence variables. *arXiv preprint arXiv:1705.00907* (2017).

[38] Nikolaos Kyrtatas, Daniele G Spampinato, and Markus Püschel. 2015. A basic linear algebra compiler for embedded processors. In *2015 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1054–1059. https://doi.org/10.7873/DATE.2015.0182

[39] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. 2020. ALBERT: A Lite BERT for Self-supervised Learning of Language Representations. In *International Conference on Learning Representations*.

[40] Chris Lattner, Jacques Pienaar, Mehdi Amini, Uday Bondhugula, River Riddle, Albert Cohen, Tatiana Shpeisman, Andy Davis, Nicolas Vasilache, and Oleksandr Zinenko. 2020. MLIR: A Compiler Infrastructure for the End of Moore's Law. *arXiv preprint arXiv:2002.11054* (2020).

[41] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. 2014. Microsoft coco: Common objects in context. In *European conference on computer vision*. Springer, 740–755.

[42] Wei Liu, Dragomir Anguelov, Dumitru Erhan, Christian Szegedy, Scott Reed, Cheng-Yang Fu, and Alexander C Berg. 2016. Ssd: Single shot multibox detector. In *European conference on computer vision*. Springer, 21–37. https://doi.org/10.1007/978-3-319-46448-0_2

[43] Lingxiao Ma, Zhiqiang Xie, Zhi Yang, Jilong Xue, Youshan Miao, Wei Cui, Wenxiang Hu, Fan Yang, Lintao Zhang, and Lidong Zhou. 2020. Rammer: Enabling Holistic Deep Learning Compiler Optimizations with rTasks. In *OSDI 2020*. USENIX Association, 881–897.

[44] Nimrod Megiddo and Vivek Sarkar. 1997. Optimal weighted loop fusion for parallel programs. In *Proceedings of the ninth annual ACM symposium on Parallel algorithms and architectures*. 282–291.

[45] Naums Mogers, Valentin Radu, Lu Li, Jack Turner, Michael O'Boyle, and Christophe Dubach. 2020. Automatic generation of specialized direct convolutions for mobile GPUs. In *Proceedings of the 13th Annual Workshop on General Purpose Processing using Graphics Processing Unit*. 41–50. https://doi.org/10.1145/3366428.3380771

[46] Wei Niu, Xiaolong Ma, Sheng Lin, Shihao Wang, Xuehai Qian, Xue Lin, Yanzhi Wang, and Bin Ren. 2020. Patdnn: Achieving real-time DNN execution on mobile devices with pattern-based weight pruning. In *ASPLOS 2020*. 907–922. https://doi.org/10.1145/3373376.3378534

[47] ONNX. 2017. *Open Neural Network Exchange*. https://www.onnx.ai

[48] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. (2019), 8024–8035.

[49] Louis-Noël Pouchet, Cédric Bastoul, Albert Cohen, and John Cavazos. 2008. Iterative optimization in the polyhedral model: Part II, multidimensional time. *ACM SIGPLAN Notices* 43, 6 (2008), 90–100.

[50] Louis-Noël Pouchet, Uday Bondhugula, Cédric Bastoul, Albert Cohen, Jagannathan Ramanujam, Ponnuswamy Sadayappan, and Nicolas Vasilache. 2011. Loop transformations: convexity, pruning and optimization. *ACM SIGPLAN Notices* 46, 1 (2011), 549–562.

[51] Benoît Pradelle, Benoît Meister, Muthu Baskaran, Jonathan Springer, and Richard Lethin. 2017. Polyhedral optimization of TensorFlow computation graphs. In *Programming and Performance Visualization Tools*. Springer, 74–89. https://doi.org/10.1007/978-3-030-17872-7_5

[52] Qualcomm. 2016. Snapdragon Profiler. https://developer.qualcomm.com/software/snapdragon-profiler

[53] Qualcomm. 2018. Snapdragon 855. https://www.qualcomm.com/products/snapdragon-855-mobile-platform

[54] Qualcomm. 2019. Snapdragon 865. https://www.qualcomm.com/products/snapdragon-865-5g-mobile-platform

[55] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language models are unsupervised multitask learners. *OpenAI blog* 1, 8 (2019), 9.

[56] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. 2013. Halide: A Language and Compiler for Optimizing Parallelism, Locality, and Recomputation in Image Processing Pipelines. In *PLDI 2013* (Seattle, Washington, USA). Association for Computing Machinery, New York, NY, USA, 519–530.

[57] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. 2015. Faster r-cnn: Towards real-time object detection with region proposal networks. In *Advances in neural information processing systems*. 91–99.

[58] Mary M Rodgers, Vinay M Pai, and Richard S Conroy. 2014. Recent advances in wearable sensors for health monitoring. *IEEE Sensors Journal* 15, 6 (2014), 3119–3126.

[59] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. 2015. U-net: Convolutional networks for biomedical image segmentation. In *International Conference on Medical image computing and computer-assisted intervention*. Springer, 234–241.

[60] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108* (2019).

[61] Manuel Selva, Fabian Gruber, Diogo Sampaio, Christophe Guillon, Louis-Noël Pouchet, and Fabrice Rastello. 2019. Building a Polyhedral Representation from an Instrumented Execution: Making Dynamic Analyses of Nonaffine Programs Scalable. *ACM Transactions on Architecture and Code Optimization (TACO)* 16, 4 (2019), 1–26.

[62] Karen Simonyan and Andrew Zisserman. 2014. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556* (2014).

[63] Khurram Soomro, Amir Roshan Zamir, and Mubarak Shah. 2012. UCF101: A dataset of 101 human actions classes from videos in the wild. *arXiv preprint arXiv:1212.0402* (2012).

[64] Daniele G Spampinato and Markus Püschel. 2014. A basic linear algebra compiler. In *CGO'14*. 23–32. https://doi.org/10.1145/2544137.2544155

[65] Zhiqing Sun, Hongkun Yu, Xiaodan Song, Renjie Liu, Yiming Yang, and Denny Zhou. 2019. MobileBERT: Task-Agnostic Compression of BERT by Progressive Knowledge Transfer. (2019).

[66] Mingxing Tan and Quoc V Le. 2019. Efficientnet: Rethinking model scaling for convolutional neural networks. *arXiv preprint arXiv:1905.11946* (2019).

[67] TensorFlow. 2017. *TensorFlow XLA*. https://www.tensorflow.org/xla

[68] TensorFlow. 2018. *TensorFlow Grappler*. https://www.tensorflow.org/guide/graph_optimization

[69] Du Tran, Lubomir Bourdev, Rob Fergus, Lorenzo Torresani, and Manohar Paluri. 2015. Learning spatiotemporal features with 3d convolutional networks. In *ICCV 2015*. 4489–4497.

[70] Nicolas Vasilache, Oleksandr Zinenko, Theodoros Theodoridis, Priya Goyal, Zachary DeVito, William S Moses, Sven Verdoolaege, Andrew Adams, and Albert Cohen. 2018. Tensor comprehensions: Framework-agnostic high-performance machine learning abstractions. *arXiv preprint arXiv:1802.04730* (2018).

[71] Anand Venkat, Mary Hall, and Michelle Strout. 2015. Loop and data transformations for sparse matrix code. *ACM SIGPLAN Notices* 50, 6 (2015), 521–532. https://doi.org/10.1145/2737924.2738003

[72] Anand Venkat, Manu Shantharam, Mary Hall, and Michelle Mills Strout. 2014. Non-affine extensions to polyhedral code generation. In *Proceedings of Annual IEEE/ACM International Symposium on Code Generation and Optimization*. 185–194.

[73] Saining Xie, Chen Sun, Jonathan Huang, Zhuowen Tu, and Kevin Murphy. 2018. Rethinking spatiotemporal feature learning: Speed-accuracy trade-offs in video classification. In *ECCV 2018*. 305–321.

[74] Tomofumi Yuki, Vamshi Basupalli, Gautam Gupta, Guillaume Iooss, D Kim, Tanveer Pathan, Pradeep Srinivasa, Yun Zou, and Sanjay Rajopadhye. 2012. Alphaz: A system for analysis, transformation, and code generation in the polyhedral equational model. *Colorado State University, Tech. Rep* (2012).